



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

GCC PLUGIN FOR STATIC ANALYZER SUPPORT

GCC PLUGIN PRO STATICKÉ ANALYZÁTORY

BACHELOR'S THESIS

BAKALÁŘSKÁ PRÁCE

AUTHOR

AUTOR PRÁCE

LUKÁŠ PŠEJA

SUPERVISOR

VEDOUCÍ PRÁCE

Dr. Ing. PETR PERINGER,

BRNO 2026

Bachelor's Thesis Assignment



172017

Institut: Department of Intelligent Systems (DITS)
Student: **Pšeja Lukáš**
Programme: Information Technology
Title: **GCC plugin for static analyzer support**
Category: Software analysis and testing
Academic year: 2025/26

Assignment:

1. Get acquainted with *CodeListener* component from the *Predator* project. Analyze the implementation of *CodeListener* and similar projects. Additionally, study the relevant data structure visualization concepts.
2. Design a new GCC plugin as a modern alternative to *CodeListener* and add new functionality (built-in visualization support and export in JSON format). Focus on improvement of the plugin interface.
3. Implement the plugin in C++ along with appropriate examples for demonstration purposes. Verify the plugin's compatibility by using it with *Predator*.
4. Test the plugin using the created examples with various GCC versions. Evaluate obtained results.

Literature:

- GitHub repository of Predator project.
<https://github.com/kdudka/predator>

Requirements for the semestral defence:

- First two points of assignment.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Peringer Petr, Dr. Ing.**
Head of Department: Kočí Radek, Ing., Ph.D.
Beginning of work: 1.11.2025
Submission deadline: 13.5.2026
Approval date: 3.11.2025

Abstract

This bachelor's thesis deals with the analysis of the Code Listener infrastructure, which serves as the foundation for creating static analyzers, and its subsequent refactoring into a new plugin.

Abstrakt

Tato bakalářská práce se zabývá analýzou infrastruktury Code Listener, která poskytuje základ pro tvorbu statických analyzátorů zdrojového kódu, a její následnou refaktorizací do nového pluginu.

Keywords

Code Listener infrastructure, GCC, LLVM, plugin, refactoring, static analysis, C, C++

Klíčová slova

infrastruktura Code Listener, GCC, LLVM, plugin, refaktorování, statická analýza, C, C++

Reference

PŠEJA, Lukáš. *GCC plugin for static analyzer support*. Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Dr. Ing. Petr Peringer,

GCC plugin for static analyzer support

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Dr. Ing. Petr Peringer. I have listed all the literary sources, publications and other sources, which were used during the preparation of this thesis. I have used Grammarly to correct spelling and other language mistakes.

.....

Lukáš Pšeja
November 29, 2025

Contents

List of Figures

Chapter 1

Introduction

With the ever-increasing complexity of software systems and the growing demand for high-quality and secure code, formal verification methods have become essential in some parts of the software development industry [?]. Formal verification methods are a set of mathematical techniques used to prove the correctness of software systems, going beyond traditional testing and debugging methods, which only test a finite number of inputs, allowing potentially critical errors to go undetected [?]. The implementation of such formal methods is not straightforward, as it requires a deep understanding of both the verified software and the application of formal methods, typically necessitating specialized domain knowledge. Formal verification tools, such as Predator or Forester [?, ?], utilize static analysis to verify code written by developers. In Predator’s case, this involves verifying sequential C programs for memory safety while operating with pointers and linked lists [?]. Predator is built on top of the Code Listener infrastructure [?], which provides an application programming interface (API) foundation for creating such static analyzers.

However, the core of the Code Listener infrastructure was implemented nearly fifteen years ago as a part of Predator, which has led to certain challenges in continuing its development and maintenance in its current form. Consequently, the codebase relies on outdated programming practices and lacks the expressiveness, safety, and efficiency offered by modern C++ standards. Moreover, this legacy technical debt, combined with the complexity of the existing codebase, hinders further development and makes maintenance and adaptation to newer GCC versions more difficult, as well as limits the potential for reusability in other static analysis tools.

To address these challenges, this bachelor’s thesis focuses on refactoring the Code Listener infrastructure into a standalone, modern GCC plugin [?]. The primary objective is to design and implement a new architecture that serves as a robust alternative to the existing solution. By leveraging the capabilities of modern C++ standards, targeting C++23, the new implementation aims to simplify the internal structure, unify the API, and ensure overall long-term maintainability. Furthermore, the modernized structure will allow for easier integration of new functionality, specifically the support for data structure visualization and export to JSON format.

The thesis is structured as follows. Chapter ?? provides an overview of the necessary background information,

Chapter 2

Preliminaries

To understand the work presented in this thesis, it is essential to have a foundational understanding of several key concepts described in this chapter.

2.1 Static program analysis

Static program analysis (expand later)

2.2 GCC

GCC

2.2.1 GCC Compilation Process

GCC Compilation Process

2.2.2 GCC plugins

GCC plugins

2.3 Predator

Predator (expand later)

2.4 Control Flow Graph

Control Flow Graph

2.5 Three Address Code

Three Address Code

2.6 The Code Listener Infrastructure

The Code Listener infrastructure was designed to help with the implementation of static analyzers for C programs. The main purpose of the Code Listener is to communicate with GCC during the compilation process, this allows it to handle everything that GCC does while compiling a program [?], translate the internal representation of the program being compiled, normalize it, and provide it to the static analyzer for further analysis in the form of an API called Code Storage or optional outputs, more on them later in this section. This is visualized in figure

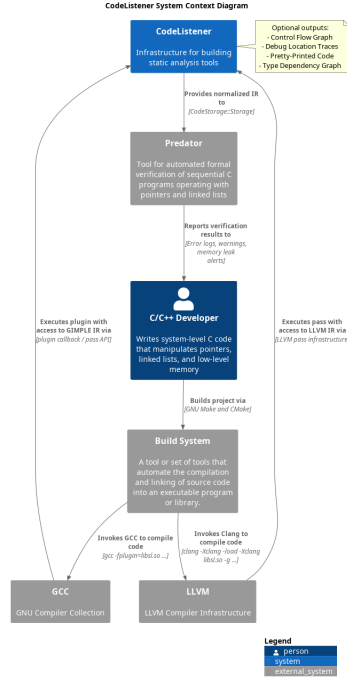


Figure 2.1: Code Listener system context diagram

The Code Listener infrastructure consists of several components visualized in figure , that work together to achieve the aforementioned goals.

Adapters are the boxes that are embedded into the code parsers (e.g., GCC or Sparse) and are responsible for translating the internal representation (IR) of the specific code parser into a normalized format, which provides a consistent way to represent the compiled program regardless of the code parser used.

The *code parser interface*, which represents the API used for communication between GCC or other code parsers such as Sparse (boxes on the left) and Code Listener during the compilation process. The API is based on callbacks which the adapters use to emit constructs of the intermediate code during traversal of the parsers's internal data structures. Behind it, there are the *filters* and *listeners*.

Filters are responsible for making transformations of the intermediate code emitted by the adapters, such as normalization or optimization and emit it to the code storage. The *switch to if* filter is an example of such a filter, which transforms switch statements intermediate code **SWITCH** into a sequence of if-else intermediate code **COND** instructions.

In contrast to filters, listeners use the API only as an input, install handlers for specific callbacks and process the intermediate code inside them. Some examples of listeners are the optional outputs, such as *control flow graph (CFG) plotter*, debug location tracer, pretty printer for three address code or type dependency graph visualizer.

Finally, there is the *code storage*, which builds the persistent object model of the intermediate code emitted by the adapters and transformed by the filters. When the whole object is built, it can be queried by the static analyzer for further analysis

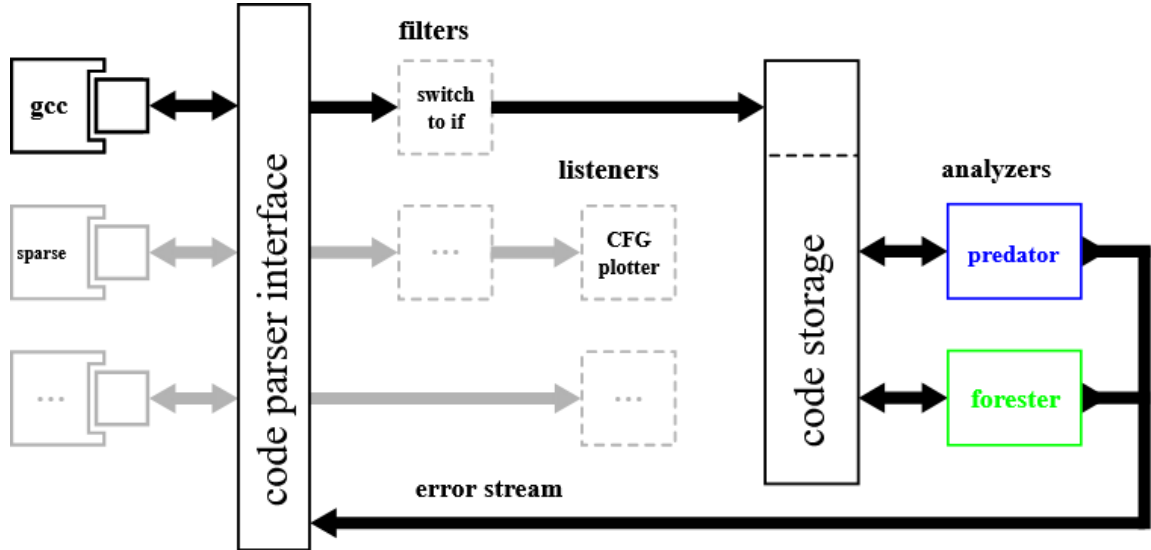


Figure 2.2: Code Listener infrastructure ¹

2.7 Modern C++ Practices

Modern C++ Practices

Chapter 3

Design

design

Chapter 4

Implementation

implementation

Chapter 5

Testing

testing

Chapter 6

Conclusion

conclusion